

Алгоритмы и структуры данных

2025–2026 учебный год

SET 9. Домашняя работа

Строки–2. Сортировка. Кодирование

май						
				1	2	3
4	5	6	7	8	9	10
11	12	13	14	15	16	17
18	19	20	21	22	23	24
25	26	27	28	29	30	31
пн	вт	ср	чт	пт	сб	вс

Немного инструкций

Задания в рамках домашней работы SET 9 подразделяются на два блока:

1. *Блок А «Аналитические задания»* — задача на экспериментальное исследование адаптированных алгоритмов сортировки строк. Решение этой задачи сопряжено с отправкой реализаций алгоритмов сортировки в систему CODEFORCES.

Решения заданий Блока А оформляются в письменном виде в любом допустимом формате и загружаются для оценки в соответствующую форму раздела SET 6 на странице курса в LMS по ссылке: <https://edu.hse.ru/course/section.php?id=1406035>.

2. *Блок Р «Задания на разработку»* — задачи, связанные с реализацией алгоритмов кодирования и декодирования строковых данных.

Решения заданий Блока Р загружаются в систему CODEFORCES и проходят автоматизированное тестирование. Для загрузки нужно перейти на <https://dsahse25.contest.codeforces.com> и выбрать соответствующее соревнование. Доступ к соревнованию предоставлен по тем же учетным данным, что и к системе Яндекс.Контест.

Домашняя работа SET 9 содержит 4(8) обязательных задач. Баллы, которые можно получить за решение задач, распределены следующим образом:

Блок А					Блок Р		
A1	A1m	A1q	A1r	A1rq	P1	P2	P3
38					7	8	8

Задачи, помеченные “b” не являются обязательными — баллы за их решение относятся к *бонусным*. Подтверждение решений бонусных задач сопровождается обязательной *устной защитой*.

Важные даты

1. Домашняя работа SET 6 открыта с **19:00 11 мая 2026 г.**
2. Прием решений завершается в **02:00 25 мая 2026 г.**

Содержание

Задача A1	Анализ строковых сортировок	1
Задача A1m	Реализация STRING MERGE SORT	4
Задача A1q	Реализация STRING QUICK SORT	5
Задача A1r	Реализация MSB RADIX SORT	6
Задача A1rq	Реализация MSB RADIX+QUICK SORT	7
Задача P1	Префиксный код Хаффмана	8
Задача P2	Кодирование LZW	9
Задача P3	Декодирование LZW	10

Успехов!

Задача А1. Анализ строковых сортировок

Стандартная сложность алгоритмов сортировки, которые работают на основе сравнений, составляет $\Omega(n \cdot \log n)$. Однако при сортировке строк эта оценка требует корректировки, поскольку приходится выполнять сравнение строк посимвольно. Для эффективной сортировки строк используются специальные методы, основанные на работе с префиксами строк. Алгоритмы учитывают совпадающие и различающиеся начальные части строк, что позволяет избежать лишних посимвольных сравнений при сортировке.

Вам необходимо провести практическое исследование, в ходе которого нужно сравнить работу обычных и модифицированных алгоритмов сортировки для массивов строк. Сортировка должна выполняться в прямом лексикографическом порядке.

- Реализация алгоритмов должна быть выполнена на языке программирования C++.
- Для обработки и визуализации полученных эмпирических результатов можно использовать любые доступные инструменты.

Этап 1. Подготовка тестовых данных

Разработайте класс `StringGenerator` для генерации массивов случайных строк. При генерации будут использоваться следующие символы (всего 74 знака):

- заглавные и строчные латинские буквы `A..Z, a..z`;
- цифры `0123456789`;
- дополнительные специальные символы `!@#%&';~&*()-.`

Параметры генерации определяются следующим образом:

- длина отдельных строк: от 10 до 200 символов;
- размер массивов: от 100 до 3000 строк с шагом 100.

Для тестирования требуется подготовить три типа массивов:

- *случайные* — полностью неупорядоченные наборы строк;
- *обратно* отсортированные — строки расположены в обратном лексикографическом порядке;
- *почти* отсортированные — создаются путем небольшого количества перестановок пар строк в полностью отсортированном массиве.

Для удобства подготовки тестовых данных, сгенерируйте для каждого случая массив максимальной длины (3000), из которого выбирайте подмассив необходимого размера (100, 200, 300, ... строк). Приветствуется генерация специальных массивов, строки в которых имеют заданные вами совпадающие префиксы.

Этап 2. Эмпирический анализ стандартных алгоритмов сортировки

Необходимо выполнить измерение производительности стандартных алгоритмов сортировки `QUICKSORT` и `MERGESORT` на подготовленных тестовых данных. При этом нужно зафиксировать:

- время выполнения алгоритмов;
- количество посимвольных сравнений (другие типы сравнений не учитываются).

Этап 3. Эмпирический анализ адаптированных алгоритмов сортировки

Требуется провести аналогичные измерения для следующих специализированных алгоритмов сортировки строк:

- тернарный **STRING QUICKSORT**;
- **STRING MERGESORT**, использующий длину наибольшего общего префикса;
- **MSD RADIX SORT** без переключения на **STRING QUICKSORT**;
- **MSD RADIX SORT** с переключением на **STRING QUICKSORT** в случае, если длина сортируемого подмассива становится меньше мощности исходного алфавита.

Важные условия выполнения Этапа 3:

- Необходимо сравнить полученные результаты с теоретическими оценками сложности адаптированных алгоритмов.
- При реализации быстрой сортировки способ выбора опорного элемента может быть любым, но должен быть одинаковым для стандартной и специализированной версии.
- Требуется провести сравнение полученных результатов с показателями стандартных алгоритмов сортировки.

Помимо графиков и пояснений, приложите:

1. Реализации классов **StringGenerator** и **StringSortTester**.
2. ID своих посылок по задачам **A1m**, **A1q**, **A1r** и **A1rq** в системе **CodeForces** с реализациями адаптированных алгоритмов сортировки строк.
3. Ссылку на публичный репозиторий с исходными данными, полученными в результате эмпирических замеров времени работы алгоритмов.

Система оценки

1. 16 баллов Реализация и успешное прохождение тестирования адаптированных алгоритмов сортировки строк в системе **CodeForces**.
2. 8 баллов Создание внутренней инфраструктуры для экспериментального анализа — классы **StringGenerator** и **StringSortTester**.
3. 7 баллов Представление эмпирических замеров производительности алгоритмов.
4. 7 баллов Проведение сравнительного анализа полученных экспериментальных данных.

Обратите внимание, что загрузка реализаций адаптированных алгоритмов сортировки строк в систему **CodeForces** является необходимым условием для получения оценки по другим критериям.

Если результаты анализа производительности алгоритмов сортировки строк не предоставлены, оценка будет выставляться по следующим правилам:

1. 10 баллов При выполнении только первого критерия (реализация и загрузка адаптированных алгоритмов в **CodeForces**).
2. 15 баллов При выполнении первого и второго критериев (реализация алгоритмов + создание инфраструктуры).

Замечание

1. Правила измерения времени работы:

- Обеспечьте минимальное влияние фоновых процессов и сетевого подключения на результаты измерений.
- Выполните многократные замеры времени работы алгоритмов и рассчитайте среднее значение (конкретное количество замеров и метод усреднения выбираете самостоятельно).
- Правильно подберите единицы измерения времени — слишком крупные единицы (например, секунды) могут дать трудно интерпретируемые результаты.

2. Создайте отдельный класс `StringSortTester` для реализации функций измерения производительности всех тестируемых алгоритмов сортировки строк.

3. Для реализации алгоритма `STRING MERGESORT` используйте материалы конспекта из раздела «Неделя 30» в системе LMS.

Измерения производительности алгоритмов должны быть организованы в рамках класса `StringSortTester`, что обеспечит структурированность кода и удобство проведения экспериментов.

Задача A1m. Реализация STRING MERGE SORT

Имя входного файла: стандартный ввод
Имя выходного файла: стандартный вывод
Ограничение по времени: 5 секунд
Ограничение по памяти: 256 мегабайт

Эта подзадача *сопутствует* основной задаче A1.

Загрузите реализацию алгоритма MERGE SORT, который адаптирован для сортировки массива строк путем подсчета длин наибольших общих префиксов `lcpCompare`.

Формат входных данных

В первой строке входных данных содержится одно целое число n ($0 \leq n \leq 3000$) — размер массива строк.

Далее приводятся n строк, длина которых находится в пределах от 10 до 200 символов. Строки образованы символами латинского алфавита A..Z, a..z, цифрами 0123456789, а также другими специальными символами. Каждый элемент исходного массива задается с новой строки.

Формат выходных данных

Выведите элементы отсортированного массива. Каждый элемент отсортированного массива выводится с новой строки.

Система оценки

Подзадача	Характеристика массивов	Необходимые подзадачи	Информация о проверке
0	тест из условия	—	полная
1	случайный массив	0	первая ошибка
2	отсортированный массив	0–1	первая ошибка
3	обратно отсортированный массив	0–2	первая ошибка

Пример

стандартный ввод	стандартный вывод
5 ac ab aa ae ad	aa ab ac ad ae

Замечание

Эта задача нацелена *только на проверку работоспособности* разработанного алгоритма.

Задача A1q. Реализация STRING QUICK SORT

Имя входного файла: стандартный ввод
Имя выходного файла: стандартный вывод
Ограничение по времени: 5 секунд
Ограничение по памяти: 256 мегабайт

Эта подзадача *сопутствует* основной задаче A1.

Загрузите реализацию тернарного алгоритма STRING QUICK SORT для сортировки массива строк.

Формат входных данных

В первой строке входных данных содержится одно целое число n ($0 \leq n \leq 3000$) — размер массива строк.

Далее приводятся n строк, длина которых находится в пределах от 10 до 200 символов. Строки образованы символами латинского алфавита A..Z, a..z, цифрами 0123456789, а также другими специальными символами. Каждый элемент исходного массива задается с новой строки.

Формат выходных данных

Выведите элементы отсортированного массива. Каждый элемент отсортированного массива выводится с новой строки.

Система оценки

Подзадача	Характеристика массивов	Необходимые подзадачи	Информация о проверке
0	тест из условия	—	полная
1	случайный массив	0	первая ошибка
2	отсортированный массив	0–1	первая ошибка
3	обратно отсортированный массив	0–2	первая ошибка

Пример

стандартный ввод	стандартный вывод
5	aa
ac	ab
ab	ac
aa	ad
ae	ae
ad	

Замечание

Эта задача нацелена *только на проверку работоспособности* разработанного алгоритма.

Задача A1r. Реализация MSB RADIX SORT

Имя входного файла: стандартный ввод
Имя выходного файла: стандартный вывод
Ограничение по времени: 5 секунд
Ограничение по памяти: 256 мегабайт

Эта подзадача *сопутствует* основной задаче A1.

Загрузите реализацию алгоритма RADIX SORT, который выполняет сортировку строк, начиная со старшего (первого) символа.

В рамках этой задачи переключение на STRING QUICK SORT не выполняется.

Формат входных данных

В первой строке входных данных содержится одно целое число n ($0 \leq n \leq 3000$) — размер массива строк.

Далее приводятся n строк, длина которых находится в пределах от 10 до 200 символов. Строки образованы символами латинского алфавита A..Z, a..z, цифрами 0123456789, а также другими специальными символами. Каждый элемент исходного массива задается с новой строки.

Формат выходных данных

Выведите элементы отсортированного массива. Каждый элемент отсортированного массива выводится с новой строки.

Система оценки

Подзадача	Характеристика массивов	Необходимые подзадачи	Информация о проверке
0	тест из условия	—	полная
1	случайный массив	0	первая ошибка
2	отсортированный массив	0–1	первая ошибка
3	обратно отсортированный массив	0–2	первая ошибка

Пример

стандартный ввод	стандартный вывод
5	aa
ac	ab
ab	ac
aa	ad
ae	ae
ad	

Замечание

Эта задача нацелена *только на проверку работоспособности* разработанного алгоритма.

Задача A1rq. Реализация MSB RADIX+QUICK SORT

Имя входного файла: стандартный ввод
Имя выходного файла: стандартный вывод
Ограничение по времени: 5 секунд
Ограничение по памяти: 256 мегабайт

Эта подзадача *сопутствует* основной задаче A1.

Загрузите реализацию алгоритма RADIX SORT, который выполняет сортировку строк, начиная со старшего (первого) символа.

В рамках этой задачи должно выполняться переключение на тернарный алгоритм STRING QUICK SORT, если на очередном шаге рекурсивного разбиения длина фрагмента массива стала меньше 74 — мощности рассматриваемого алфавита.

Формат входных данных

В первой строке входных данных содержится одно целое число n ($0 \leq n \leq 3000$) — размер массива строк.

Далее приводятся n строк, длина которых находится в пределах от 10 до 200 символов. Строки образованы символами латинского алфавита A..Z, a..z, цифрами 0123456789, а также другими специальными символами. Каждый элемент исходного массива задается с новой строки.

Формат выходных данных

Выведите элементы отсортированного массива. Каждый элемент отсортированного массива выводится с новой строки.

Система оценки

Подзадача	Характеристика массивов	Необходимые подзадачи	Информация о проверке
0	тест из условия	—	полная
1	случайный массив	0	первая ошибка
2	отсортированный массив	0–1	первая ошибка
3	обратно отсортированный массив	0–2	первая ошибка

Пример

стандартный ввод	стандартный вывод
5	aa
ac	ab
ab	ac
aa	ad
ae	ae
ad	

Замечание

Эта задача нацелена *только на проверку работоспособности* разработанного алгоритма.

Задача Р1. Префиксный код Хаффмана

Имя входного файла: стандартный ввод
Имя выходного файла: стандартный вывод
Ограничение по времени: 1 секунда
Ограничение по памяти: 128 мегабайт

В дверь постучали 1024 раза.

— Килобайт, — подумал Штирлиц.
— Открывай уже, чёрт побери! — подумал Мюллер.

Профессор Плейшнер должен передать Штирлицу тайное сообщение с помощью письма так, чтобы немецкие шпионы не догадались о его содержании. Для этого, будучи учёным человеком, он решил использовать префиксный код Хаффмана.

По данной строке s , состоящей из строчных букв латинского алфавита, постройте ее оптимальный префиксный код с помощью кодирования Хаффмана.

Формат входных данных

В первой строке входных данных непустая строка s ($1 \leq |s| \leq 10^6$) - текст, который пытается зашифровать Плейшнер.

Формат выходных данных

В первой строке выведите количество различных букв k , встречающихся в строке, и размер получившейся закодированной строки.

В следующих k строках выведите коды букв в формате «буква: код». В последней строке выведите полученный код исходной строки.

Система оценки

Подзадача	Баллы	Дополнительные ограничения	Необходимые подзадачи	Информация о проверке
0	—	тесты из условия	—	полная
1	1	$ s \leq 1000$	0	первая ошибка
2	4	$ s \leq 2 \cdot 10^4$	0–1	первая ошибка
3	2	$ s \leq 10^6$	0–2	первая ошибка

Пример

стандартный ввод	стандартный вывод
rendezvous	9 32 d: 1110 e: 110 n: 1111 o: 000 r: 001 s: 010 u: 011 v: 100 z: 101 00111011111110110101100000011010

Задача Р2. Кодирование LZW

Имя входного файла: стандартный ввод
Имя выходного файла: стандартный вывод
Ограничение по времени: 2 секунды
Ограничение по памяти: 128 мегабайт

В дверь постучали
В дверь постучали
В дверь постучали
В дверь постучали

«Пакетов: отправлено = 4, получено = 0,
потеряно = 4 (100% потерь)» – подумал
Штирлиц.

Штирлиц выяснил, что один из высокопоставленных немецких чиновников тайно ведёт переговоры с американцами. Ему нужно срочно сообщить об этом Центру, но для эффективной передачи сообщение должно быть сжато. Опытный разведчик выбрал алгоритм LZW. Помогите ему!

Изначально словарь кодов содержит 128 значений (0–127 в десятичном формате) для отдельных ASCII-символов с соответствующими кодами.

Формат входных данных

Ввод содержит строку s ($1 \leq |s| \leq 10^4$), состоящую из символов ASCII, — исходное сообщение Штирлица Центру.

Формат выходных данных

В первой строке выведите одно целое число k — количество фрагментов, кодирующих сообщение в соответствии с алгоритмом LZW.

Во второй строке выведите k целых чисел через пробел — фрагменты LZW-кода в десятичной записи.

Система оценки

Подзадача	Баллы	Дополнительные ограничения	Необходимые подзадачи	Информация о проверке
0	—	тесты из условия	—	полная
1	1	$ s \leq 100$	0	первая ошибка
2	2	$ s \leq 1000$	0–1	первая ошибка
3	2	$ s \leq 9 \cdot 10^3$	0–2	первая ошибка
4	3	$ s \leq 10^4$	0–3	первая ошибка

Примеры

стандартный ввод	стандартный вывод
Universal algo.	14 85 110 105 118 101 114 115 97 108 32 135 103 111 46
Science? Science!	14 83 99 105 101 110 99 101 63 32 128 130 132 101 33
Schellenberg	12 83 99 104 101 108 108 101 110 98 101 114 103

Задача Р3. Декодирование LZW

Имя входного файла: стандартный ввод
Имя выходного файла: стандартный вывод
Ограничение по времени: 2 секунды
Ограничение по памяти: 128 мегабайт

– Штирлиц, а почему у Вас шесть
пальцев? – вкрадчиво поинтересовался
Мюллер.
Никогда ИИ не был так близко к провалу.

Штирлиц получил ответ от Центра на своё сообщение. По классике советской разведки, оно тоже
сжато с помощью алгоритма LZW.

Выполните декодирование полученного текстового сообщения.

Изначально словарь содержит 128 значений (0–127) для отдельных ASCII-символов с соответ-
ствующими кодами.

Формат входных данных

Первая строка содержит целое число k ($1 \leq k \leq 10^4$) — количество фрагментов, кодирующих
сообщение из Центра в соответствии с алгоритмом LZW.

Вторая строка содержит k целых чисел c_i ($0 \leq c_i \leq 10^4$) — кодирующие фрагменты в десятичной
записи.

Формат выходных данных

Выведите одну строку — исходное сообщение.

Система оценки

Подзадача	Баллы	Дополнительные ограничения	Необходимые подзадачи	Информация о проверке
0	–	тесты из условия	–	полная
1	1	$k \leq 10$	0	первая ошибка
2	1	$k \leq 100$	0–1	первая ошибка
3	3	$k \leq 1000$	0–2	первая ошибка
4	3	$k \leq 10^4$	0–3	первая ошибка

Примеры

стандартный ввод	стандартный вывод
14 85 110 105 118 101 114 115 97 108 32 135 103 111 46	Universal algo.
14 83 99 105 101 110 99 101 63 32 128 130 132 101 33	Science? Science!
12 83 99 104 101 108 108 101 110 98 101 114 103	Schellenberg